



MODUL 9 · DATABASE DENGAN PRISMA 7

# *Filtering, Sorting, Pagination, Relasi & Transaksi*

Query yang powerful — cari, urutkan, paginate, dan mutasi multiple table dengan aman.



# Filtering — Operator yang Perlu Diketahui

Prisma menyediakan operator filter yang powerful dan type-safe.

● ● ● Prisma where operators — cheat sheet

```
// — Comparison operators —————
await db.post.findMany({ where: {
  viewCount: { gte: 100 },           // >= 100
  viewCount: { lte: 1000 },          // <= 1000
  viewCount: { gt: 0 },              // > 0
  createdAt: { gte: new Date("2025-01-01") },
}})

// — String operators —————
await db.post.findMany({ where: {
  title:    { contains: "Next.js", mode: "insensitive" }, // case-insensitive
  slug:     { startsWith: "nextjs" },
  content:  { endsWith: "conclusion" },
}})

// — Logical operators —————
await db.post.findMany({ where: {
  AND: [{ published: true }, { viewCount: { gte: 10 } }],
  OR:  [{ featured: true }, { viewCount: { gte: 100 } }],
  NOT: { deletedAt: null },
}})

// — Array/Relation operators —————
await db.post.findMany({ where: {
  tags: { some: { name: "nextjs" } }, // ada tag 'nextjs'
  tags: { every: { active: true } },  // semua tag active
  tags: { none: { deprecated: true } }, // tidak ada tag deprecated
  id:   { in: ["id1", "id2", "id3"] }, // ID dalam array
}})
```

# Pagination — Cursor-based & Offset

Dua strategi: offset (skip/take) untuk UI, cursor (cursor/take) untuk infinite scroll.

● ● ● Offset pagination (page number)

```
// — Offset pagination —————
// Cocok untuk halaman bernomor (page 1, 2, 3...)
const PAGE_SIZE = 10

async function getPosts(page: number) {
  const [posts, total] = await db.$transaction([
    db.post.findMany({
      where: { published: true },
      orderBy: { createdAt: "desc" },
      take: PAGE_SIZE,
      skip: (page - 1) * PAGE_SIZE,
      select: { id: true, title: true, slug: true },
    }),
    db.post.count({ where: { published: true } }),
  ])

  return {
    posts,
    total,
    totalPages: Math.ceil(total / PAGE_SIZE),
    currentPage: page,
    hasNext: page * PAGE_SIZE < total,
    hasPrev: page > 1,
  }
}
```

● ● ● Cursor pagination (infinite scroll)

```
// — Cursor pagination
// Cocok untuk infinite scroll / load more
// Lebih efisien untuk dataset besar

async function getPostsCursor(cursor?: string) {
  const posts = await db.post.findMany({
    where: { published: true },
    orderBy: { createdAt: "desc" },
    take: 11, // ambil 11 untuk cek hasMore
    // cursor: gunakan ID dari record terakhir
    cursor: cursor ? { id: cursor } : undefined,
    skip: cursor ? 1 : 0, // skip cursor itu sendiri

    select: { id: true, title: true, slug: true },
  })

  const hasMore = posts.length > 10
  const data = hasMore ? posts.slice(0, 10) : posts
  const nextCursor = hasMore ? data[9].id : null

  return { posts: data, nextCursor, hasMore }
}
```

# Transactions — Semua atau Tidak Sama Sekali

Groupkan multiple operasi DB. Jika satu gagal, semua di-rollback.

```
// — $transaction — sequential operations —————
// Semua dijalankan dalam satu transaction. Satu gagal → rollback semua.
const [updatedUser, newPost] = await db.$transaction([
  db.user.update({
    where: { id: userId },
    data: { postCount: { increment: 1 } },
  }),
  db.post.create({
    data: { title, slug, content, authorId: userId },
  }),
])
// Jika post.create gagal → user.update juga di-rollback!

// — $transaction dengan function (lebih flexible) —————
const result = await db.$transaction(
  async (tx) => {
    // tx = transaction client — pakai ini, bukan db!
    const post = await tx.post.create({ data: { title, authorId } })

    // Tambahkan ke audit log
    await tx.auditLog.create({
      data: { action: 'POST_CREATED', postId: post.id, userId: authorId },
    })

    // Kirim notifikasi ke followers
    const followers = await tx.follow.findMany({
      where: { followingId: authorId },
    })
    await tx.notification.createMany({
      data: followers.map((f) => ({ userId: f.followerId, postId: post.id })),
    })

    return post
  },
  {
    timeout: 10000, // max 10 detik
    maxWait: 5000, // max tunggu koneksi
    isolationLevel: 'Serializable', // isolasi penuh
  }
)
```

P R I S M A 7 & P O S T G R E S Q L

# Seeding — Isi Database dengan Data Awal

Seed file untuk test data, demo data, dan initial configuration.

```
import { db } from '../lib/db'
// Atau langsung buat client di seed:
// import { PrismaClient } from "../generated/prisma/client"
// const db = new PrismaClient()

async function main() {
  console.log('🚀 Seeding database...')

  // Hapus data lama (urutan: child dulu, parent belakangan)
  await db.comment.deleteMany()
  await db.post.deleteMany()
  await db.user.deleteMany()

  // Buat users
  const admin = await db.user.create({
    data: { email: 'admin@example.com', name: 'Admin', role: 'ADMIN' },
  })
  const author = await db.user.create({
    data: { email: 'author@example.com', name: 'Author One', role: 'AUTHOR' },
  }, {})

  // Buat posts dengan relasi
  await db.post.createMany({
    data: [
      {
        title: 'Menenal Next.js 16',
        slug: 'next-js-16',
        published: true,
        authorId: author.id,
      },
      {
        title: 'Prisma 7 Tutorial',
        slug: 'prisma-7',
        published: true,
        authorId: author.id,
      },
    ],
  })

  console.log('✅ Seeding complete!')
}

main()
  .catch((e) => {
    console.error(e)
    process.exit(1)
  })
  .finally(() => db.$disconnect())

// Jalankan: bunx prisma migrate dev (auto) atau tsx prisma/seed.ts (manual)
```

# Evaluasi Pemahaman

Jawab sebelum lanjut.

**Q1 — Untuk search case-insensitive di Prisma PostgreSQL, apa yang perlu ditambahkan?**

- A) contains: 'keyword' saja (default insensitive)      B) contains: 'keyword', mode: 'insensitive'      C) ilike: 'keyword'

**Q2 — Apa keuntungan cursor pagination dibanding offset untuk dataset besar?**

- A) Bisa lompat ke halaman berapa saja      B) Tidak perlu scan rows yang dilewati — lebih efisien untuk dataset > 100k      C) Lebih mudah diimplementasikan

**Q3 — Di \$transaction dengan function, mengapa harus pakai 'tx' bukan 'db' untuk query?**

- A) 'db' deprecated di dalam transaction      B) Query dengan 'db' berada di luar transaction scope — tidak akan di-rollback jika gagal      C) Hanya masalah naming convention

# Tugas Mandiri

Implementasi filtering, pagination, dan seed data.

## 01 Blog Search

Buat `getPosts({ query, category, page })` yang pakai `where contains` + `mode insensitive` untuk search, dan `skip/take` untuk pagination.

## 02 Cursor Paginate

Implementasikan `getPostsCursor(cursor?)` untuk 'Load More' button. Return `{ posts, nextCursor, hasMore }`.

## 03 Transaction

Buat `createPostWithNotification(data)` yang dalam satu transaction: create post + create notification untuk semua followers author.

## 04 Seed File

Buat `prisma/seed.ts` dengan: 3 users (1 admin, 2 authors), 10 posts, 20 comments tersebar. Jalankan: `tsx prisma/seed.ts`.



Test transaction: tambahkan `throw new Error('test')` setelah create post, verifikasi post tidak terbuat.

# Yang Sudah Kita Pelajari

- ✓ Filter operators: contains + mode insensitive, gte/lte/gt, in, some/every/none untuk relasi.
- ✓ Offset pagination: skip/take + count. Cursor pagination: lebih efisien untuk dataset besar.
- ✓ Transaction: semua berhasil atau rollback. Pakai 'tx' bukan 'db' dalam function-style transaction.
- ✓ Seed: hapus child sebelum parent. Configure di prisma.config.ts untuk auto-run dengan migrate.

→ Selanjutnya: **Bab 8 — Connection Pooling untuk Serverless & Production**

